

Reviewer Pack — Minimal Number of Non-Rigid Motions and Communication Comple...

Entry 4c55fa85a226 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 23:02:30 UTC

Minimal Number of Non-Rigid Motions and Communication Complexity Rank

Entry ID: 4c55fa85a226 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 23:02:30 UTC

1. Conjecture

Field A (mathematical branch): Rigidity Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given d -regular graph G with a communication complexity rank $r(G)$, the minimal number of non-rigid motions required to transform an isometric embedding of G into a regular grid with maximum overlap is $O(r(G)^2)$.

Rationale (proposer's reasoning):

Rigidity theory, particularly in the context of non-rigid motions, provides a framework for understanding the constraints on geometric objects. The conjecture proposes that these constraints have a direct relationship with communication complexity rank, which measures the complexity of a function in terms of its ability to be communicated between parties. This bridge could expose new insights into the nature of computation and communication.

Taxonomy category: RIGIDITY_TO_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0b363e77ba96f888

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all d -regular graphs G with varying ranks $r(G)$, the number of non-rigid motions required to transform an isometric embedding into a regular grid is at most $O(r(G)^2)$ for 80% or more of the seeds. The conjecture is falsified if, for any seed, the number of non-rigid motions exceeds $O(r(G)^2)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "rigidity theory" AND "communication complexity" AND matrix rank" - "non-rigid motion" AND regular graph" AND communication complexity" - "isometric embedding" AND rigidity theory" AND regular grid" AND communication complexity rank"

Top relevant hits considered: - [http://arxiv.org/abs/2603.09104v2] Training-free Motion Factorization for Compositional Video Generation - [http://arxiv.org/abs/1804.06023v1] DoubleFusion: Real-time Capture of Human Performances with Inner Body Shapes from a Single Depth Sensor - [http://arxiv.org/abs/1910.11414v1] Reconstruction of Undersampled 3D Non-Cartesian Image-Based Navigators for Coronary MRA Using an Unrolled Deep Learning - [http://arxiv.org/abs/2006.15423v4] A dynamical-topological obstruction for smooth isometric embeddings of Riemannian manifolds via incompressible Euler eq - [http://arxiv.org/abs/1305.5613v4] Isometric embeddings via heat kernel - [http://arxiv.org/abs/2010.12677v1] Quasi-local mass and isometric embedding with reference to a static spacetime

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_d_regular_graph(d, n):
    if d * (d - 1) // 2 < n or n % d != 0:
        raise ValueError("Invalid parameters for generating a d-regular graph")

    G = [[] for _ in range(n)]
    edges_added = set()

    def add_edge(u, v):
        if u == v or (u, v) in edges_added or (v, u) in edges_added:
            return
        G[u].append(v)
        G[v].append(u)
        edges_added.add((u, v))
        edges_added.add((v, u))

    for i in range(n):
```

```

        neighbors = random.sample(range(i + 1, n), d - 1)
        for neighbor in neighbors:
            add_edge(i, neighbor)

    return G

def isometric_embedding(G):
    n = len(G)
    embedding = [None] * n
    used_positions = set()

    def dfs(node, pos):
        if node in used_positions:
            return False
        used_positions.add(pos)
        embedding[node] = pos
        for neighbor in G[node]:
            if not dfs(neighbor, (pos[0], pos[1] + 1)):
                return False
        return True

    if not dfs(0, (0, 0)):
        return None

    return embedding

def non_rigid_transformations(embedding):
    n = len(embedding)
    grid_size = max(pos[1] for pos in embedding.values()) + 1
    grid = [[None] * grid_size for _ in range(grid_size)]

    for node, pos in embedding.items():
        grid[pos[0]][pos[1]] = node

    def find_non_rigid_moves():
        moves = []
        for i in range(grid_size):
            for j in range(grid_size):
                if grid[i][j] is not None:
                    continue
                for di in [-1, 0, 1]:
                    for dj in [-1, 0, 1]:
                        ni, nj = i + di, j + dj
                        if 0 <= ni < grid_size and 0 <= nj < grid_size and grid[ni][nj] is not None:
                            moves.append((grid[ni][nj], (ni, nj), (i, j)))

        return moves

    return find_non_rigid_moves()

def run_trial(seed: int) -> dict:
    random.seed(seed)

    d = 3
    n = 10
    G = generate_d_regular_graph(d, n)
    embedding = isometric_embedding(G)

```

```

if embedding is None:
    return {
        "metric_name": "non_rigid_moves",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "isometric_embedding_failed"
    }

non_rigid_moves = non_rigid_transformations(embedding)
num_non_rigid_moves = len(non_rigid_moves)
r_G_squared = d**2

return {
    "metric_name": "non_rigid_moves",
    "metric_value": num_non_rigid_moves,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": num_non_rigid_moves <= r_G_squared,
    "counterexample": "" if num_non_rigid_moves <= r_G_squared else f"num_non_rigid_moves={num_non_rigid_moves}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_86b82fb3.py", line 120, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_86b82fb3.py", line 88, in run_trial
    G = generate_d_regular_graph(d, n)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_86b82fb3.py", line 20, in generate_d_re
    raise ValueError("Invalid parameters for generating a d-regular graph")
ValueError: Invalid parameters for generating a d-regular graph

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that it was unable to verify the conjecture for the given seeds. | next: Investigate the cause of the crash and attempt to run the test again with appropriate parameters.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14311
2	preregistration	ollama_remote	glm4:latest	0	0	9710
3	novelty	ollama_remote	glm4:latest	0	0	17564
4	novelty	ollama_remote	glm4:latest	0	0	14811
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	139394
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25507
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13959
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13853
9	judge	ollama_remote	glm4:latest	0	0	27037

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 276148 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4c55fa85a226.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4c55fa85a226.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4c55fa85a226.tar.gz (if generated)